

Reviewer Pack — Euler Characteristic of Configuration Spaces and Communicati...

Entry 99691e96eed1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 21:45:49 UTC

Euler Characteristic of Configuration Spaces and Communication Complexity

Entry ID: 99691e96eed1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 21:45:49 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Communication Complexity

Statement:

For all Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, the Euler characteristic $\chi(f)$ of the configuration space of the associated quantum information system is $\Omega(n^{3/2})$. Furthermore, for any given constant $c > 0$ and a Boolean matrix $M \in \{\pm 1\}^{N \times N}$, there exists an n such that $\chi(M) \leq cn^{3/2}$ implies the randomized communication complexity $CC_R(M) \leq n^c$.

Rationale (proposer's reasoning):

Configuration space theory provides a geometric framework for understanding quantum information systems, which have been shown to have connections with classical communication complexity. This conjecture suggests that the Euler characteristic of the configuration space could be used as a lower bound for communication complexity, offering a new perspective on the interplay between geometry and computation.

Taxonomy category: Configuration_Space_Theoretic (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 76ef71639c02c62a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The Euler characteristic χ of configuration spaces meets $\Omega(n^{3/2})$ and for fixed $c > 0$, $\chi(M) \leq cn^{3/2}$ implies $CC_R(M) \leq n^c$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space theory" AND "communication complexity" AND Euler characteristic" - "quantum information system" AND "Euler characteristic" AND "communication complexity" - "Boolean function" AND "randomized communication complexity" AND configuration space

Top relevant hits considered: - [s2:c33660f74ad2cb933fb6b8c972e58593877e7af8] Making a Case for Spatial Prompting in Human-Robot Communication

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def euler_characteristic(f):
        n = len(f)
        if n == 0:
            return 0
        chi = 0
        for i in range(n):
            if f[i] == 1:
                chi += (-1)**i * (n - i)
        return chi

    def communication_complexity(M, c):
        N = len(M)
        rows = random.sample(range(N), N // 2)
        cols = random.sample(range(N), N // 2)
        cc = 0
        for r in rows:
            for c in cols:
                if M[r][c] == 1:
                    cc += 1
        return cc / (N * N) ** c
```

```

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    f = generate_boolean_function(n)
    chi_f = euler_characteristic(f)
    if chi_f < 0:
        continue
    total_metric_value += chi_f
    instances_tested += 1
    n_max = max(n_max, n)

    # Check communication complexity for a random Boolean matrix M
    M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    cc_M = communication_complexity(M, 0.5)
    if cc_M > chi_f ** (2/3):
        conjecture_holds = False
        counterexample = f"CC_R(M)={cc_M} > {chi_f**(2/3)}"

return {
    "metric_name": "Euler Characteristic",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15321
2	preregistration	ollama_remote	glm4:latest	0	0	19102
3	novelty	ollama_remote	glm4:latest	0	0	8556
4	novelty	ollama_remote	glm4:latest	0	0	12314
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13798
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14111
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31476
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9823
9	judge	ollama_remote	glm4:latest	0	0	47284

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 171786 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/99691e96eed1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/99691e96eed1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/99691e96eed1.tar.gz` (if generated)